

Tavily MCP Server: Complete Guide (English)

Written so you can understand the project from zero: the idea, the problem, how it was solved, the technologies, the API, and what every command used along the way does. Arabic version: [GUIDE.ar.md](#)

Contents

- 1. [The idea and where it came from](#)
 - 2. [Problem and solution](#)
 - 3. [What MCP is and how the project works](#)
 - 4. [Technologies used](#)
 - 5. [The Tavily API](#)
 - 6. [File layout and code walkthrough](#)
 - 7. [The four tools and prompts to try them](#)
 - 8. [Every command used, explained](#)
 - 9. [Publishing to GitHub: what to allow and what to block](#)
 - 10. [Problems we hit and lessons](#)
 - 11. [Security and key management](#)
-

1. The idea and where it came from

AI models learn from data up to a cutoff date and have no live internet access by themselves. Ask about today's news, a current price, or a new page and they may answer from stale memory.

The idea: **give the assistant a live web-search tool** through a small server that talks to **Tavily**, a search service built for AI applications (it returns clean, ranked content instead of raw HTML).

Honest note: this section explains the general motivation as visible from the code and goals (live search inside the desktop client and the CLI client). No file in the repository records the project's origin story; if you have a personal one, add it here.

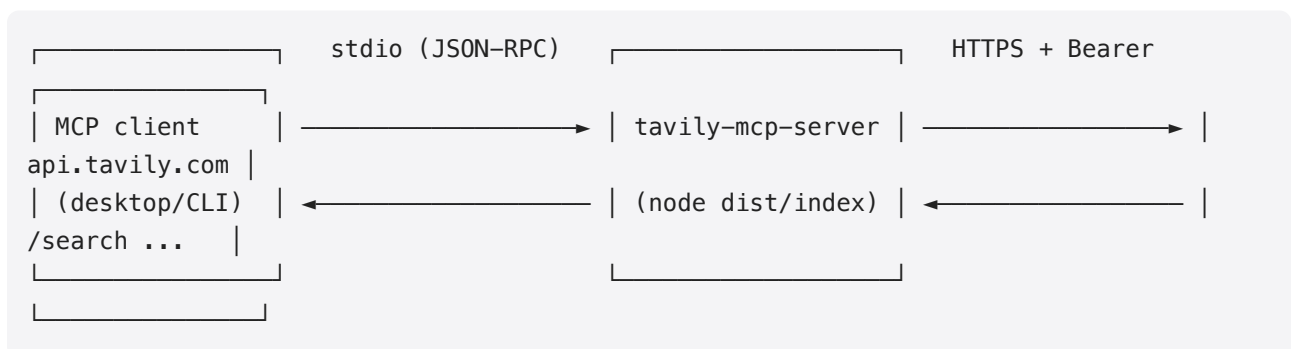
2. Problem and solution

Problem	Solution in this project
The model can't see the internet; its knowledge is old	<code>tavily_search</code> searches live and returns sourced results

Problem	Solution in this project
Ordinary search results are noisy HTML	Tavily returns clean text with a relevance <code>score</code>
We need a whole page, not a snippet	<code>tavily_extract</code> pulls the content of specific URLs
We need a site's structure	<code>tavily_map</code> lists URLs, <code>tavily_crawl</code> fetches content
Every app connects to tools differently	MCP standardizes it (the desktop client, the CLI client, Cursor)
Huge responses eat the model's context	<code>truncate</code> cuts replies at 25,000 characters with a notice
Cryptic API errors	<code>explainHttpError</code> maps status codes to clear messages
Key leaks	The key comes from an environment variable; <code>.env</code> stays out of Git

3. What MCP is and how the project works

MCP (Model Context Protocol) is an open protocol that lets AI apps (clients) use external tools (servers) in a uniform way.



Flow for each search: 1. The desktop client launches `node dist/index.js` as a child process (from its config file). 2. They talk over **stdio** with **JSON-RPC** messages: `initialize`, `tools/list`, `tools/call`. 3. When you ask for a search, the client sends `tools/call` for `tavily_search`. 4. Our server validates input (zod) and sends `POST https://api.tavily.com/search` with the key. 5. It formats the reply as Markdown (or JSON) and returns it, truncating if long.

Important: logs go to `stderr` (`console.error`) because `stdout` belongs to the MCP protocol; writing anything else there corrupts the connection.

4. Technologies used

Technology	Version (installed here)	Role
Node.js	22.x (required $\geq$ 18)	Runtime for the main implementation
TypeScript	5.9.x	Source language, compiled to JavaScript in <code>dist/</code>
<code>@modelcontextprotocol/sdk</code>	1.32.x	Official MCP library (<code>McpServer</code> , <code>StdioServerTransport</code>)
<code>zod</code>	3.25.x	Input validation and parameter descriptions
Built-in <code>fetch</code>	—	Calls Tavily (no extra HTTP library)
<code>tsx</code>	4.x	Run TypeScript directly in development (<code>npm run dev</code>)
Python	3.12	The simpler alternative <code>server.py</code>
<code>mcp</code> (FastMCP)	< 2	MCP library for Python
<code>tavily-python</code>	$\geq$ 0.5.0	Official Tavily Python client
<code>python-dotenv</code>	$\geq$ 1.0.0	Reads <code>.env</code> in the Python version
<code>uv</code>	—	Fast creation of a Python 3.12 environment (macOS system Python was 3.9)
Git and GitHub CLI (<code>gh</code>)	—	Version control and publishing

Why two implementations? `src/` (TypeScript) is the full one: four tools, strict validation, formatted output. `server.py` is a minimal single-tool version (`web_search`), useful for learning and comparison. The one wired into the desktop client is the **TypeScript** version.

5. The Tavily API

- **Base URL:** `https://api.tavily.com`
- **Method:** `POST` with a JSON body
- **Auth:** header `Authorization: Bearer tvly-...`
- **Get a key:** `https://app.tavily.com`
- **Timeout in our code:** `180 seconds (AbortSignal.timeout(180_000))`

Endpoint	Used by tool	Purpose
<code>/search</code>	<code>tavily_search</code>	Web search
<code>/extract</code>	<code>tavily_extract</code>	Extract content from URLs (up to 20)
<code>/map</code>	<code>tavily_map</code>	Discover a site's URLs, no content
<code>/crawl</code>	<code>tavily_crawl</code>	Crawl a site and return content

Key `/search` parameters: `query`, `search_depth` (`basic` | `advanced` | `fast` | `ultra-fast`), `topic` (`general` | `news` | `finance`), `max_results` (1-20), `time_range` (`day` / `week` / `month` / `year`), `start_date` / `end_date` (`YYYY-MM-DD`), `include_answer`, `include_raw_content`, `include_images`, `include_domains` / `exclude_domains`, `country`.

Error codes we translate:

Code	Meaning	Our message
400	Invalid parameters	Check parameter values
401	Bad key	Verify the key starts with <code>tvly-</code>
429	Rate limited	Wait and send fewer requests
432 / 433	Plan or credit limit exceeded	Check usage at <code>app.tavily.com</code>

Cost: `search_depth="advanced"` costs 2 credits (per the code's description). `crawl` and advanced `extract` cost more. See Tavily's official pricing for credit details.

6. File layout and code walkthrough

```

.
├─ src/
│   ├── index.ts      # server + the four tool definitions
│   └─ client.ts      # Tavily calls, error mapping, truncation
├─ dist/              # compiled output (not committed)
└─ server.py          # Python alternative (web_search tool)
```

```

├─ requirements.txt      # Python dependencies
├─ package.json         # Node dependencies and npm scripts
├─ tsconfig.json        # TypeScript compiler settings
├─ .env                 # real key (never committed)
├─ .env.example         # key-less template (committed)
├─ .gitignore           # what must not be committed
├─ mcp_client_config.example.json # sample MCP client config
├─ README.md / README.ar.md
└─ docs/GUIDE.ar.md / GUIDE.en.md      # this guide

```

src/client.ts

- `BASE_URL` and `CHARACTER_LIMIT = 25000`.
- `tavilyPost(endpoint, body)`: reads `TAVILY_API_KEY` from `process.env` and throws a clear error if missing. It drops `undefined` values so Tavily applies its own defaults, sends the `POST`, and translates any non-OK response via `explainHttpError`.
- `truncate(text)`: cuts text at 25,000 characters and appends a hint to narrow the request.

src/index.ts

- Creates an `McpServer` named `tavily-mcp-server`.
- `readOnly`: annotations telling the client these tools are read-only (`readOnlyHint`) and non-destructive, so they change nothing on your machine.
- `responseFormat`: `markdown` (default, readable) or `json` (raw response).
- `ok()` builds the reply (text + `structuredContent`); `fail()` returns `isError: true` with a clear message instead of crashing the server.
- Each tool: `registerTool(name, {description, zod inputSchema, outputSchema, annotations}, handler)`.
- Finally: `StdioServerTransport`, then `server.connect(transport)`.

server.py (alternative)

One tool, `web_search`. It validates the key, `query`, `max_results` (1-20) and `search_depth`. The Tavily client is synchronous, so it runs inside `asyncio.to_thread` to avoid blocking the event loop. It returns JSON with `title`, `url`, `content`, or `{"error": ...}`. It loads `.env` from beside the file regardless of the working directory.

7. The four tools and prompts to try them

Paste into a **new chat** in the desktop client and name the tool so the assistant doesn't use its built-in search.

tavily_search - Use `tavily_search` to search "what's new in TypeScript" and give me 5 results - Use `tavily_search` with `topic=news` and `time_range=week` for AI news, with `include_answer=true` - Use `tavily_search` for "MCP server examples" with `include_domains=["github.com"]` and `max_results=3`

tavily_extract - Use `tavily_extract` on `https://modelcontextprotocol.io/introduction` and summarize it - ... with `query="what is MCP"` and `chunks_per_source=2`

tavily_map (cheapest: URLs only) - Use `tavily_map` on `https://modelcontextprotocol.io` and list the URLs

tavily_crawl (most expensive, start small) - Use `tavily_crawl` on `https://modelcontextprotocol.io` with `max_depth=1` and `limit=5`, then summarize - ... with `instructions="find pages about building servers"`

Signs of success: a permission prompt naming the tool, real links in the results, `map` returning only URLs while `crawl` returns URLs with content.

8. Every command used, explained

Split into (A) core build/run commands and (B) this session's commands in order.

A) Core build and run commands

Command	What it does
<code>npm install</code>	Reads <code>package.json</code> , downloads libraries into <code>node_modules/</code> , writes <code>package-lock.json</code> to pin versions.
<code>npm run build</code>	Runs <code>tsc</code> (the build script), compiling <code>src/*.ts</code> into <code>dist/*.js</code> per <code>tsconfig.json</code> .
<code>npm start</code>	<code>node dist/index.js</code> : runs the server manually (it waits for MCP messages on stdin).

Command	What it does
<code>npm run dev</code>	<code>tsx watch src/index.ts</code> : development mode, restarts on edits without compiling.
<code><client-cli> mcp add tavily -e TAVILY_API_KEY=... -- node .../dist/index.js</code>	Registers the server in the CLI client . <code>-e</code> passes an env var; everything after <code>--</code> is the launch command.
<code>npx @modelcontextprotocol/inspector node dist/index.js</code>	Opens the official UI for trying tools by hand. Put the key first: <code>TAVILY_API_KEY=... npx ...</code>
<code>uv venv --python 3.12 .venv</code>	Creates an isolated Python 3.12 environment (system Python here was 3.9, below the required 3.10).
<code>uv pip install --python .venv/bin/python -r requirements.txt</code>	Installs the Python libraries into that environment only.
<code>.venv/bin/mcp dev server.py</code>	Runs the Python version with the official inspector.

B) This session's commands, in order

1) Inspecting the project - `ls -la`: lists all files including hidden ones (`.env`) with permissions and sizes. - `find . -path ./node_modules -prune -o -type f -not -path './.git/*' -print | head -50`: lists files while skipping `node_modules` and `.git`; `head -50` keeps the first 50 lines. - `cat README.md ... ; sed 's/=.*/= <hidden>/' .env`: reads files. `sed` here **hides the key's value** after `=` so it never appears in the conversation. - `wc -l src/*.ts`: counts lines per file.

2) Testing both implementations for real - Python test: `.venv/bin/python - <<'EOF' ... EOF` runs Python written inline (heredoc). It imported `server` and called `web_search` via `asyncio.run` with a real query. Result: two results, no errors. - TypeScript test: `node -e '...'` runs a short script that spawns `node dist/index.js` as a child process and speaks JSON-RPC to it by hand: `initialize`, `notifications/initialized`, `tools/list`, `tools/call`. This is exactly what the desktop client does, so it proved the four tools are listed and search works. - The first run returned `TAVILY_API_KEY not set` because the TS version **does not read** `.env`. Fix: `set -a && . ./env && set +a`, then rerun. `set -a` auto-exports every variable defined next, `. ./env` loads the file into the current shell, `set +a` turns auto-export off.

3) Wiring the desktop client - Listed "`<client config file>`" and printed it with every field containing `KEY`, `TOKEN` or `SECRET` masked via `sed -E`, to see what was there before editing. - `cp "$F" "$F.bak"`: a backup before any edit. (Delete it later since it contains the key.) - `node -e '...'` to merge: extracts the key from `.env` with a regex, reads the JSON config, adds `mcpServers.tavily`, and writes it back. Merging keeps your other settings instead of replacing the file. - `chmod 600 "$F"`: makes the file readable/writable by you alone since it now holds the key. - Removing the Python entry used another `node -e` script that deletes `mcpServers["tavily-web-search"]`. - `Cmd+Q` then reopen: the desktop client reads its config only at startup.

4) Troubleshooting - `ls -t <client logs folder> | head` then `tail` on `mcp-server-tavily.log`: the log proved the server started and sent its tool list but had **no `tools/call`**, meaning the assistant used its built-in search, not our tool. We verified instead of guessing. `grep -v -i 'tvly-'` drops any line that could contain a key.

5) Cleaning Python files (restored later) - `cat .gitignore && du -sh .venv`
`__pycache__`: review what we're about to delete and its size (42 MB). - `rm -rf .venv`
`__pycache__` `server.py` `requirements.txt` ...: permanent deletion with no trash, so we inspected the target first. The files were restored at your request afterward.

6) Checking for keys before publishing - `grep -rILE 'tvly-[A-Za-z0-9_-]{10,}' .`
`--exclude-dir=node_modules`: searches for anything shaped like a Tavily key and prints filenames only. (-r recursive, -I skip binary, -l names only, -E regex.) - **A trap we learned:** -I treats a file as binary if it contains Arabic under some locales, so it skipped `.env` (which has an Arabic comment). Use -a instead when a file contains Arabic. - `grep -rILF -`
`- "$K"`: searches for the **exact real key text** (-F literal); -- stops the key being read as an option.

7) Publishing to GitHub - `git init -q -b main`: creates a local repository on branch `main` (-q quiet). - `git add -A`: stages every file **except** what `.gitignore` excludes. - `git ls-files`: shows what will actually be uploaded, the most important review point. - `git grep -aF -- "$K"`: searches for the real key inside **tracked files only**; the desired result is "no match". - `gh auth status`: confirms you're logged in to GitHub CLI. - `git commit -m "..."`: records the first snapshot. - `gh repo create tavily-mcp-server --private --source=.` --push: creates a **private** repo on your account, links the current folder (--source=.) and pushes (--push). - `git log --oneline` and `git status -sb`: confirm the commit exists and local matches remote. - Later: `sed -i '' 's#...#...#'`
`README.md` replaces text in a file on macOS (the '' is required by macOS `sed`). We replaced my personal path with a generic one.

9. Publishing to GitHub: what to allow and what to block

Allowed

File/folder	Reason
src/	Source code
package.json, package-lock.json	Pin libraries and versions for rebuilds
tsconfig.json	Compiler settings
server.py, requirements.txt	The alternative implementation
.env.example	Template with no real key
mcp_client_config.example.json	Sample with generic paths and a fake key
README.md, README.ar.md, docs/	Documentation
.gitignore	Protects against committing secrets

Blocked

File/folder	Reason
.env	Contains the real key
mcp_client_config.json (real)	May contain the key
*.bak	Config backups containing the key
node_modules/	Huge; recreated by <code>npm install</code>
dist/	Build output; recreated by <code>npm run build</code>
.venv/	Python environment (42 MB); recreated by <code>uv venv</code>
__pycache__/, .DS_Store	System/temporary files

Checklist before every `git push`

```
git status          # .env and .bak must not appear
git ls-files        # review the whole list
K=$(grep -E '^TAVILY_API_KEY=' .env | cut -d= -f2- | tr -d "\"" " ")
git grep -aF -- "$K" && echo "DANGER: key is in a tracked file!" || echo "clean"
```

If a key ever entered a past commit, deleting it from the files isn't enough (it stays in history). The only reliable fix: **revoke the key at app.tavily.com and create a new one.**

10. Problems we hit and lessons

Problem	Cause	Fix
<code>TAVILY_API_KEY</code> not set when testing TS	The TS version doesn't read <code>.env</code>	Pass the key via <code>env</code> in config or <code>set -a; . ./env</code>
The assistant didn't ask permission or use Tavily	It used built-in search	New chat, name the tool in the prompt, or disable built-in search
Server missing after editing config	the desktop client reads config at startup	<code>Cmd+Q</code> , then reopen
<code>python3.12</code> not found	System has only 3.9	<code>uv venv --python 3.12</code> fetches and sets it up
<code>grep -I</code> skipped <code>.env</code>	Arabic content treated as binary	Use <code>-a</code>
Editing the config could wipe other settings	The file holds all app settings	Merge with a script instead of replacing, plus a <code>.bak</code> backup

11. Security and key management

- The key is read from `process.env.TAVILY_API_KEY` and never written into code.
- Never put the key in any file that goes to Git.
- the desktop client's config holds the key, with permissions `600` (you only).
- All tools are read-only (`readOnlyHint`) and write nothing on your machine.
- To revoke a key: <https://app.tavily.com> → API Keys → Delete or Regenerate, then update `.env` and the desktop client's config.